Table of Contents

Delta Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

DELTA Network Specification Index

Distributed Execution Layer for Tensor Acceleration Version 0.3 - Early Proof of Concept (Draft)

Document Organization

This specification describes DELTA, a configurable mesh Network-on-Chip (NoC) that routes four distinct packet types between the RAPIDS DMA engine, HIVE control system, and compute tiles.

Chapter 1: Overview

01. Executive Summary 02. System Integration 03. Packet Type Routing 04. Acronyms and References

Chapter 2: Blocks

00. Block Overview 01. Router Architecture 02. Network Interface 03. Packet Classifier 04. Virtual Channel Allocator 05. Crossbar Switch

Chapter 3: Interfaces

01. AXIS Interface Specification 02. Packet Format and Framing 03. External Entity Integration

Chapter 4: Programming Models

01. Virtual Configuration Contexts 02. Context Switching 03. Configuration Commands 04. Complete System Flow

Chapter 5: Registers

01. Configuration Register Bank 02. Router Status Registers 03. Performance Counters

Appendices

Appendix A: Tile Coordinate Mapping Appendix B: Configuration Commands Appendix C: Performance Characteristics

---

Document Version: 0.3 (Early Draft - Proof of Concept) **Last Updated:** 2025-10-19
Status: Preliminary specification, subject to significant change **Maintained By:** DELTA Development Team

---

1. Executive Summary

DELTA is a configurable mesh Network-on-Chip (NoC) that routes four distinct packet types between the RAPIDS DMA engine, HIVE control system, and compute tiles. The network implements intelligent packet filtering and routing based on AXIS TUSER encoding, ensuring protocol isolation between data, control, and monitoring traffic.

Key Features

- 4x4 mesh topology (16 compute tiles) for proof-of-concept

- Four packet types: DATA, DESC, CONFIG, STATUS
- Virtual configuration contexts for topology reconfiguration
- Wormhole flow control with virtual channels
- XY dimension-ordered routing (deadlock-free)
- Per-tile packet filtering and forwarding rules
- AXIS protocol throughout

Design Goals

1. **Protocol Isolation**: Four packet types routed independently without interference
2. **Flexible Topologies**: Virtual contexts support mesh, systolic, tree, and custom modes
3. **Deadlock-Free**: XY routing with virtual channels guarantees forward progress
4. **Educational Focus**: Clear architecture with hooks for experimentation
5. **Integration**: Seamless connection to RAPIDS DMA and HIVE control plane

Target Applications

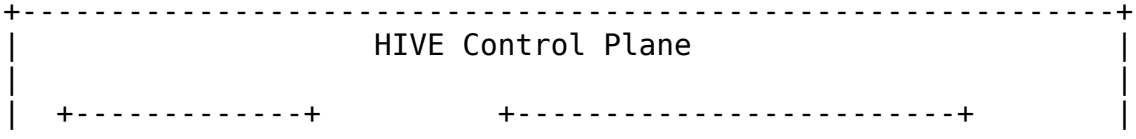
- Machine learning inference acceleration
- Tensor operations on distributed compute tiles
- Network-on-Chip research and education
- Configurable interconnect exploration

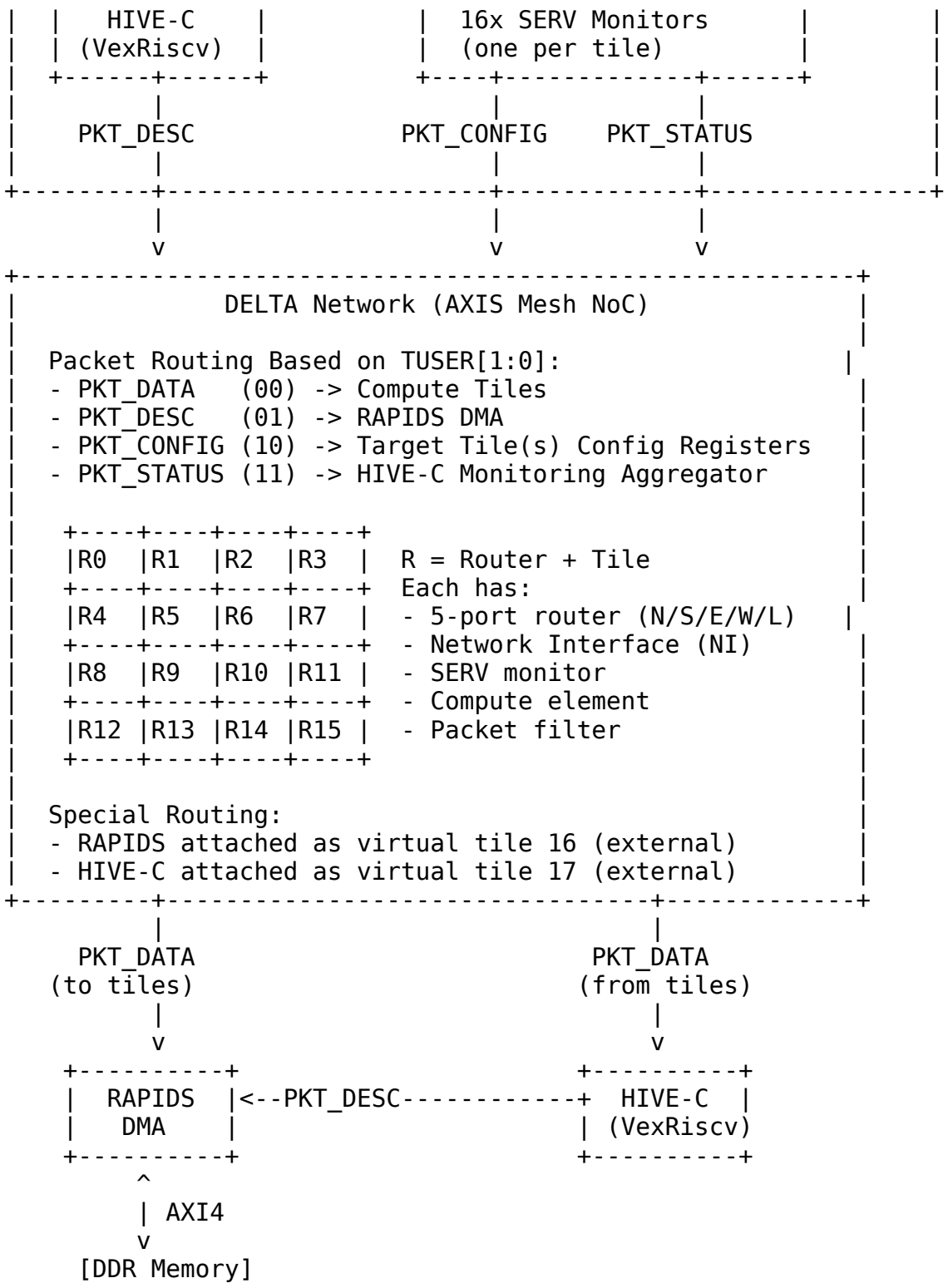
Next: System Integration

---

2. System Integration

2.1 Network Position





2.2 Packet Flow Matrix

Source	Packet Type	Destination(s)	Path Through DELTA
HIVE-C	PKT_DESC	RAPIDS	Direct route to virtual tile

Source	Packet Type	Destination(s)	Path Through DELTA
			16
HIVE-C	PKT_CONFIG	Tile(s) 0-15	Multicast via tile mask
RAPIDS	PKT_DATA	Tile(s) 0-15	XY route to TDEST tile
Tile N	PKT_DATA	RAPIDS	XY route to virtual tile 16
Tile N	PKT_DATA	Tile M	XY route peer-to-peer
SERV N	PKT_CONFIG	Tile N (local)	Direct to local config registers
SERV N	PKT_STATUS	HIVE-C	XY route to virtual tile 17

2.3 Never Occurs (Hardware Prevents)

- RAPIDS sending PKT_DESC (only receives)
- Compute Tiles sending PKT_DESC (not authorized)
- PKT_DATA routed to HIVE-C (filtered)
- PKT_CONFIG routed to RAPIDS (filtered)

Next: Packet Type Routing

---

3. Packet Type Routing

3.1 AXIS TUSER Encoding

All AXIS interfaces use this encoding:

```
// All AXIS interfaces use this encoding
typedef enum logic [1:0] {
    PKT_DATA   = 2'b00,  // Compute data (activations, weights,
results)
    PKT_DESC   = 2'b01,  // DMA descriptor (HIVE-C -> RAPIDS only)
    PKT_CONFIG = 2'b10,  // Configuration command (HIVE-C/SERV ->
Tiles)
    PKT_STATUS = 2'b11   // Status/monitoring (SERV -> HIVE-C)
} packet_type_t;
```

3.2 Routing Decision Logic

Each router examines TUSER[1:0] and TDEST[3:0]:

```
always_comb begin
    case (pkt_tuser)
        PKT_DATA: begin
            // Route to compute tile or RAPIDS
            if (pkt_tdest == 4'd16) begin
                route_to_rapids();  // Virtual tile 16
            end else if (pkt_tdest < 4'd16) begin
                route_to_tile(pkt_tdest);  // Tile 0-15
            end else begin
                drop_packet();  // Invalid destination
            end
        end

        PKT_DESC: begin
            // Always route to RAPIDS (only valid destination)
            route_to_rapids();
        end

        PKT_CONFIG: begin
            // Route to tile(s) based on mask in payload
            // Supports multicast for broadcast configs
            route_by_tile_mask();
        end

        PKT_STATUS: begin
            // Always route to HIVE-C monitoring aggregator
            route_to_hive_c();  // Virtual tile 17
        end
    endcase
end
```

3.3 Virtual Tile Mapping

DELTA extends the 4x4 physical mesh with virtual tiles:

Tile ID	Type	Location	Purpose
0-15	Physical	Mesh positions (0,0) to (3,3)	Compute tiles
16	Virtual	External (south of tile 12)	RAPIDS DMA
17	Virtual	External (north of tile 3)	HIVE-C controller

Routing to Virtual Tiles

- Tile 16 (RAPIDS): Reached via tile 12 south port
- Tile 17 (HIVE-C): Reached via tile 3 north port
- Physical routing unaffected (XY still works)

Next: Acronyms and References

---

4. Acronyms and References

Acronyms

Acronym	Definition
AXIS	Advanced eXtensible Interface Stream (AXI Stream)
DDR	Double Data Rate (memory)
DELTA	Distributed Execution Layer for Tensor Acceleration
DMA	Direct Memory Access
HIVE	Hierarchical Intelligent Verification Engine
MM2S	Memory-Mapped to Stream
NI	Network Interface
NoC	Network-on-Chip
PKT	Packet
RAPIDS	Rapid AXI Programmable In-band Descriptor System
SERV	Simple Embedded RISC-V (monitor core)
S2MM	Stream to Memory-Mapped
TID	Transaction ID (AXIS sideband

Acronym	Definition
	signal)
TDEST	Transaction Destination (AXIS sideband signal)
TUSER	Transaction User (AXIS sideband signal)
VC	Virtual Channel
XY	X-dimension then Y-dimension (routing algorithm)

References

AMBA Specifications

- AXI4-Stream Protocol Specification v1.0 (ARM IHI 0051A)

Related Components

- **RAPIDS DMA**: Rapid AXI Programmable In-band Descriptor System
  - See `projects/components/dma-ip/rapids/docs/rapids_beats_has/` and `rapids_beats_mas/`
- **HIVE Control**: VexRiscv-based control plane
  - See `projects/components/compute-eng-ip/hive-serv/` (re-homed from compute-eng-ip 2026-10-10)

Academic References

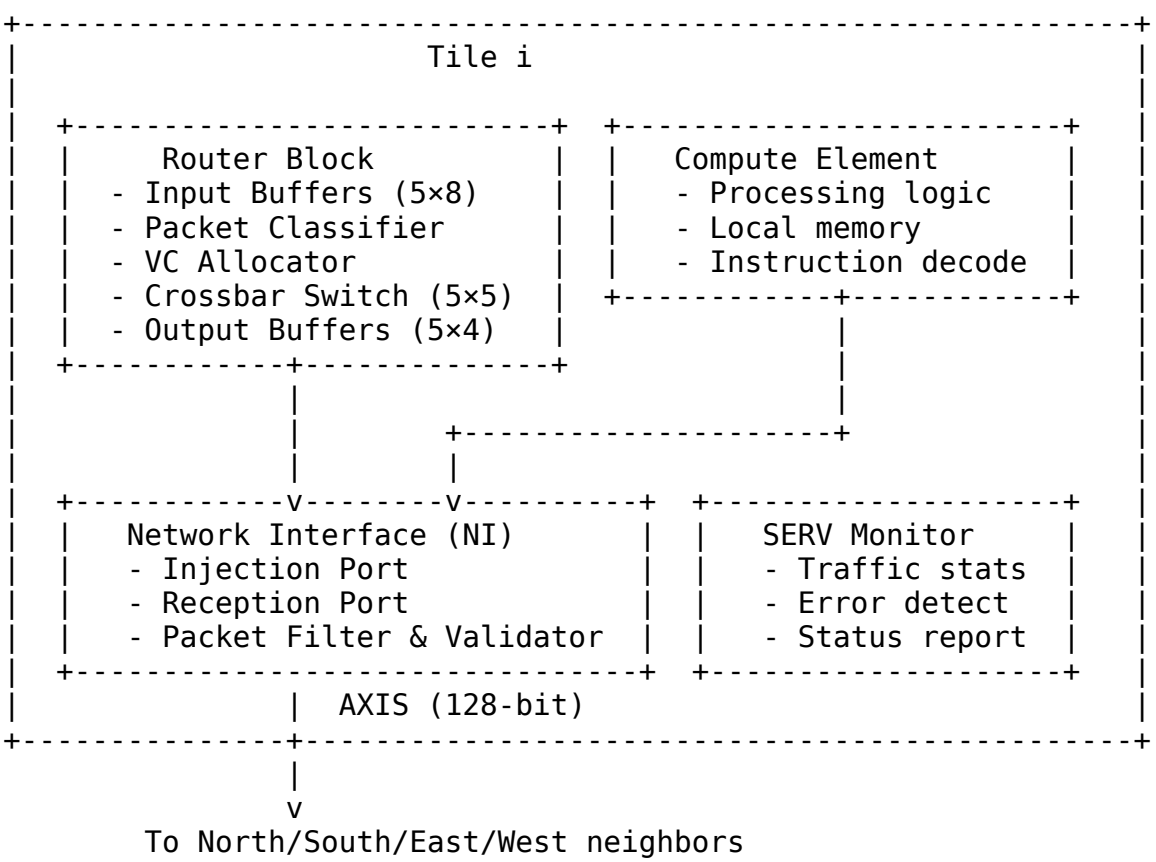
- Dally, W. J., & Towles, B. (2004). “Principles and Practices of Interconnection Networks”
- Duato, J., Yalamanchili, S., & Ni, L. (2002). “Interconnection Networks: An Engineering Approach”

Back to: Index | **Next Chapter:** Block Overview

Chapter 2: Block Overview

DELTA's 4x4 mesh consists of 16 router/tile combinations, each containing several functional blocks working together to route packets and execute computation.

Block Hierarchy



Major Functional Blocks

1. Router Block

- **Purpose**: Route packets between five ports (N/S/E/W/Local)
- **Key Components**:
  - Input buffers (8 flits per port)
  - Packet classifier (TUSER/TDEST decode)
  - Virtual channel allocator (2 VCs per link)
  - 5×5 non-blocking crossbar
  - Output buffers (4 flits per port)
- **Details**: Router Architecture

2. Network Interface (NI)

- **Purpose**: Connect compute element to router
- **Key Functions**:
  - Packetization of local data
  - Injection rate control
  - Packet type enforcement
  - Reception filtering
- **Details**: Network Interface

3. Packet Classifier

- **Purpose**: Determine routing based on packet type and destination
- **Key Functions**:
  - Read TUSER[1:0] (packet type)
  - Read TDEST[3:0] (destination tile)
  - Compute output port via XY routing
  - Handle virtual tile mapping
- **Details**: Packet Classifier

4. Virtual Channel Allocator

- **Purpose**: Arbitrate between virtual channels for output port access
- **Key Functions**:
  - Manage credit-based flow control
  - Prioritize control traffic (VC0) over data (VC1)
  - Prevent deadlock via VC separation
- **Details**: Virtual Channel Allocator

5. Crossbar Switch

- **Purpose**: Connect any input port to any output port
- **Key Functions**:
  - 5×5 non-blocking routing
  - Single-cycle traversal
  - Conflict-free simultaneous transfers
- **Details**: Crossbar Switch

Block Interaction Flow

```
Packet arrives at North input port:

1. Input Buffer receives packet
```

```
   - Stores 8 flits deep
   - Sends credit back to upstream

2. Packet Classifier examines header
   - Reads TUSER = PKT_DATA
   - Reads TDEST = 5 (tile 5)
   - Computes: current tile = 1, dest = 5
   - XY routing: go East (column 1 -> 2)

3. VC Allocator checks East output
   - PKT_DATA uses VC1
   - Credits available for VC1? Yes (6/8)
   - Grant packet to crossbar

4. Crossbar routes North -> East
   - Single cycle transfer
   - No conflicts with other ports

5. Output Buffer stages packet
   - Dequeues from East buffer
   - Sends to East neighbor
   - Returns credit to North input
```

Resource Budget (Per Router)

Component	LUTs	FFs	BRAM (36Kb)
Input Buffers (5×8 flits)	2,000	1,600	64 bits
Packet Classifier	200	150	0
VC Allocator	150	100	0
Crossbar (5×5)	400	200	0
Output Buffers (5×4 flits)	800	600	32 bits
Total per router	**~3,550**	**~2,650**	**~2**

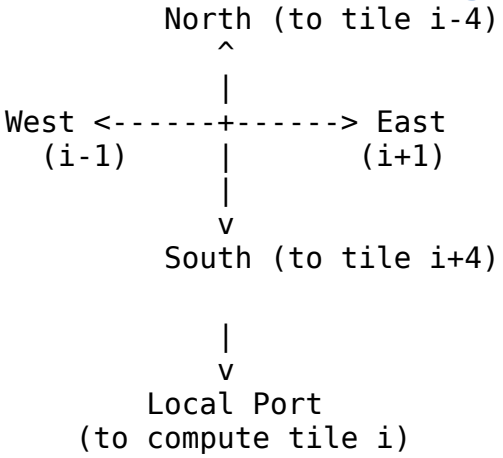
Full 4×4 Network (16 routers): - 16 × 3,550 = 56,800 LUTs - 16 × 2,650 = 42,400 FFs - 16 × 2 = 32 BRAM blocks

---

Next: Router Architecture

1. Router Architecture

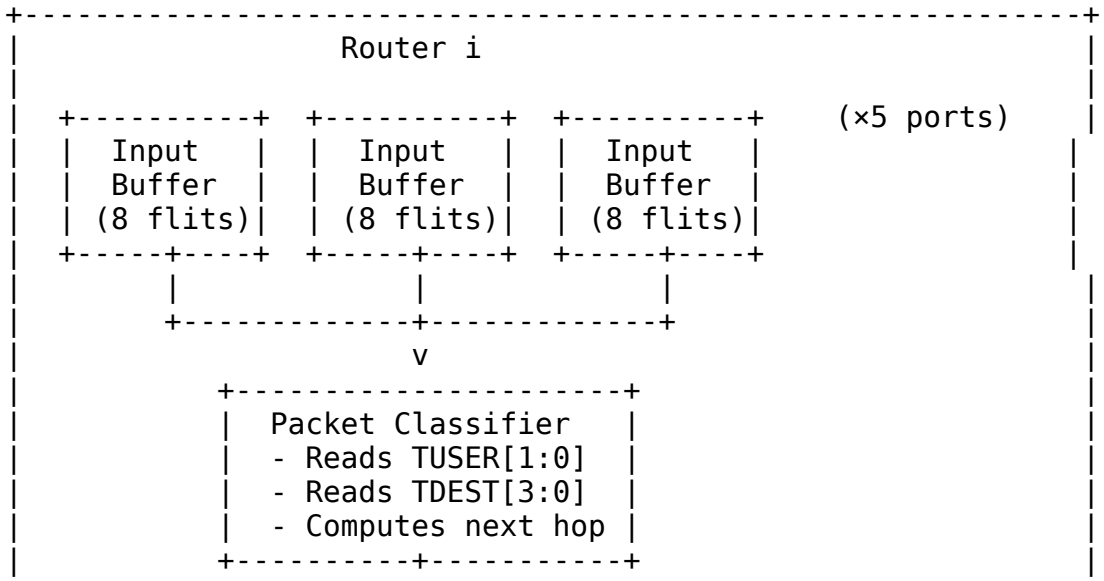
1.1 Five-Port Router Design

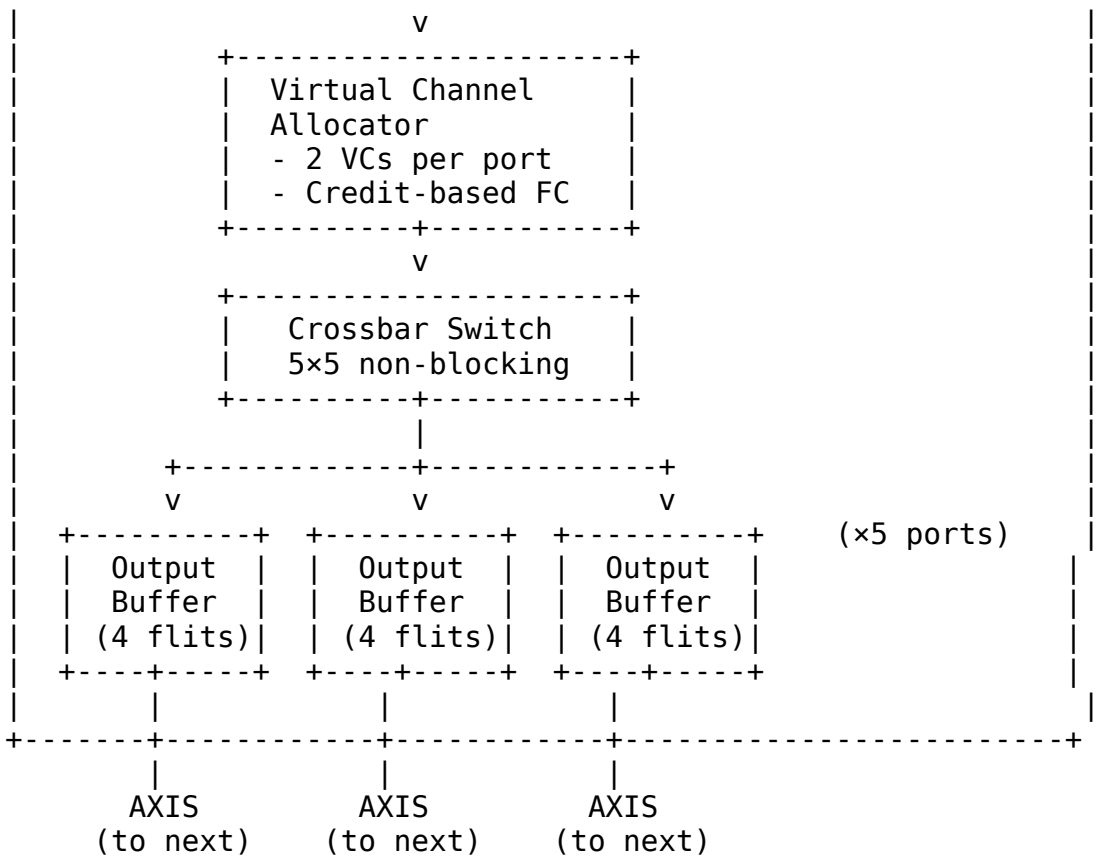


Port Functions

- **North/South/East/West:** Inter-router links (128-bit AXIS)
- **Local:** Connection to tile's Network Interface (NI)

1.2 Router Block Diagram





1.3 XY Routing Algorithm

```
function [2:0] compute_output_port(
    input [3:0] current_tile,  // 0-15 for mesh, 16=RAPIDS, 17=HIVE-C
    input [3:0] dest_tile,
    input [1:0] pkt_type
);
    logic [1:0] curr_x, curr_y;
    logic [1:0] dest_x, dest_y;

    // Extract coordinates (4x4 mesh)
    curr_x = current_tile[1:0];  // Column
    curr_y = current_tile[3:2];  // Row

    // Handle virtual tiles
    if (dest_tile == 16) begin  // RAPIDS
        dest_x = 2'd0;  // Column 0
        dest_y = 2'd4;  // Virtual row 4 (south of row 3)
    end else if (dest_tile == 17) begin  // HIVE-C
        dest_x = 2'd3;  // Column 3
        dest_y = 2'd255; // Virtual row -1 (north of row 0)
    end else begin
        dest_x = dest_tile[1:0];
        dest_y = dest_tile[3:2];
```

```
    end

    // XY routing: X-dimension first
    if (dest_x < curr_x) begin
        return PORT_WEST;
    end else if (dest_x > curr_x) begin
        return PORT_EAST;
    end else if (dest_y < curr_y) begin
        return PORT_NORTH;
    end else if (dest_y > curr_y) begin
        return PORT_SOUTH;
    end else begin
        return PORT_LOCAL;  // Arrived at destination
    end
endfunction
```

1.4 Routing Latency

Single-hop latency: - Input buffering: 1 cycle - Routing decision: 1 cycle - Crossbar traversal: 1 cycle - Output buffering: 1 cycle (optional) - **Total: 3-4 cycles per hop**

Multi-hop examples (4×4 mesh): | Source -> Dest | Hops | Latency (cycles) | | ——————-|——|——————| | T0 -> T1 | 1 | 3-4 | | T0 -> T5 | 2 | 6-8 | | T0 -> T15 | 6 | 18-24 (worst case) | | Any -> RAPIDS (avg) | 3.5 | 10-14 | | Any -> HIVE-C (avg) | 3.5 | 10-14 |

Next: Network Interface

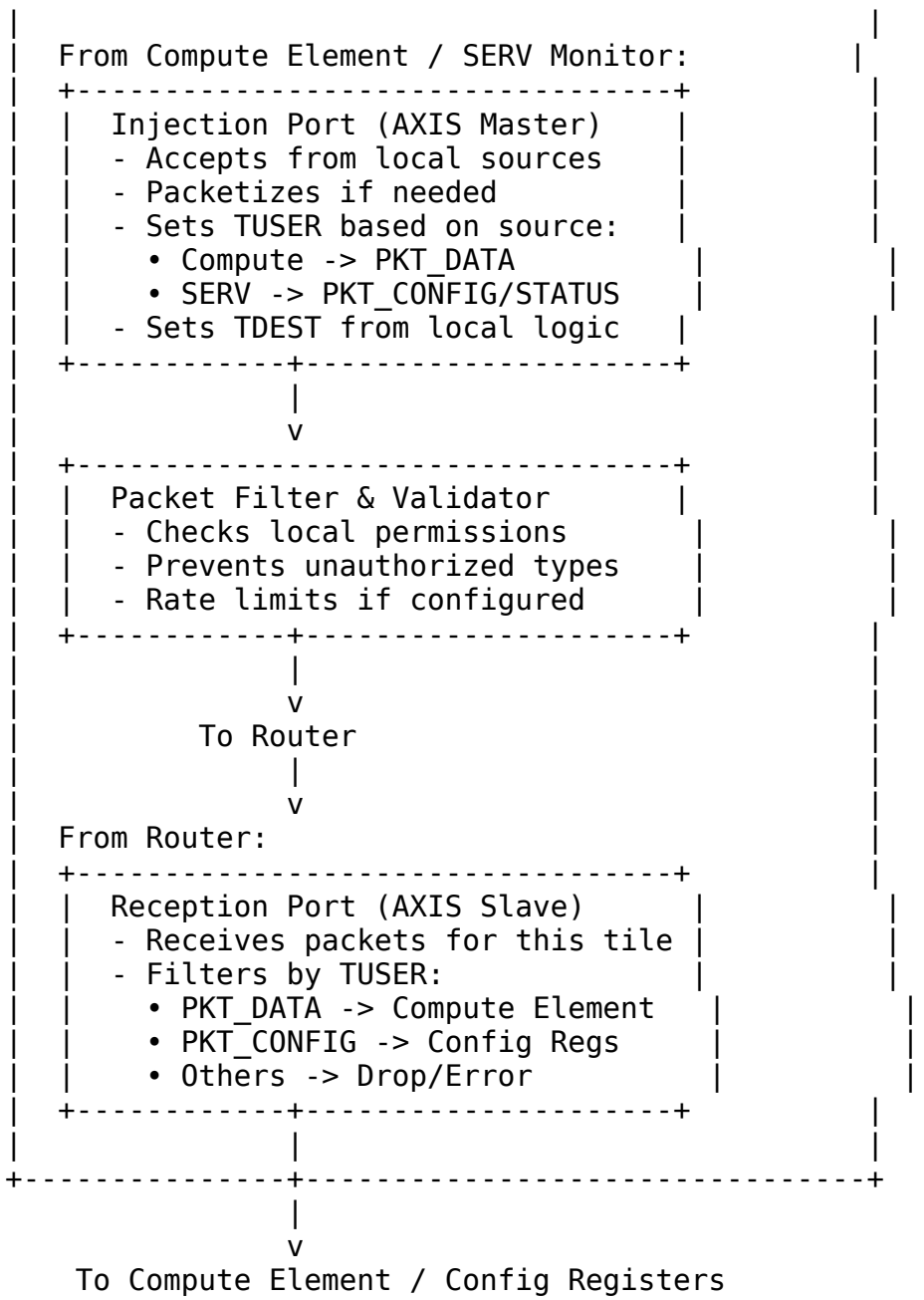
---

2. Network Interface (NI)

2.1 Per-Tile Network Interface

Each tile has a Network Interface connecting the compute element to the router:

```
+-----------------------------------------------+
|         Network Interface (Tile i)            |
```



2.2 Packet Filtering Rules

Each NI enforces these rules:

Injection (Tile -> Network)

```
// Compute elements can only send PKT_DATA
if (source == COMPUTE && pkt_tuser != PKT_DATA) begin
    reject_packet();
    set_error_flag(ERR_UNAUTHORIZED_TYPE);
end
```

```
// CRITICAL: Compute elements MUST set TID to own tile ID
// This allows RAPIDS to route incoming data to correct S2MM channel
if (source == COMPUTE && pkt_tuser == PKT_DATA) begin
    // Hardware enforces TID = MY_TILE_ID for data from compute
    pkt_tid_override = MY_TILE_ID[3:0];
end

// SERV monitors can send PKT_CONFIG (local) or PKT_STATUS (to HIVE-C)
if (source == SERV && !(pkt_tuser inside {PKT_CONFIG, PKT_STATUS}))
begin
    reject_packet();
    set_error_flag(ERR_UNAUTHORIZED_TYPE);
end
```

Reception (Network -> Tile)

```
case (pkt_tuser)
    PKT_DATA: begin
        // Route to compute element
        forward_to_compute();
    end

    PKT_CONFIG: begin
        // Route to configuration registers
        forward_to_config_regs();
    end

    PKT_DESC, PKT_STATUS: begin
        // Not valid for regular tiles
        drop_packet();
        set_error_flag(ERR_INVALID_DEST);
    end
endcase
```

2.3 TID Enforcement

CRITICAL: Network Interface Hardware Enforcement

The TID field serves different purposes based on packet type. The NI hardware enforces correct TID values:

Packet Type	TID Meaning	Set By	Enforced By NI
PKT_DATA (to RAPIDS)	Source Tile ID (0-15)	NI (forced)	YES - override
PKT_DATA (to Tiles)	Priority/Sequence	RAPIDS MM2S	NO - pass through
PKT_DESC	Priority Level	HIVE-C	NO - pass through

Packet Type	TID Meaning	Set By	Enforced By NI
PKT_CONFIG	Broadcast ID	HIVE-C/ SERV	NO - pass through
PKT_STATUS	Reporting Tile ID	NI (forced)	YES - override

TID Override Logic:

```
// Network Interface enforces TID for specific packet types
always_comb begin
    if (pkt_source == LOCAL_COMPUTE && pkt_tuser == PKT_DATA) begin
        // Override TID to this tile's ID
        pkt_tid_out = MY_TILE_ID[3:0];
    end else if (pkt_source == LOCAL_SERV && pkt_tuser == PKT_STATUS)
begin
        // Override TID to this tile's ID for status reports
        pkt_tid_out = MY_TILE_ID[3:0];
    end else begin
        // Pass through original TID
        pkt_tid_out = pkt_tid_in;
    end
end
```

Next: Packet Classifier

---

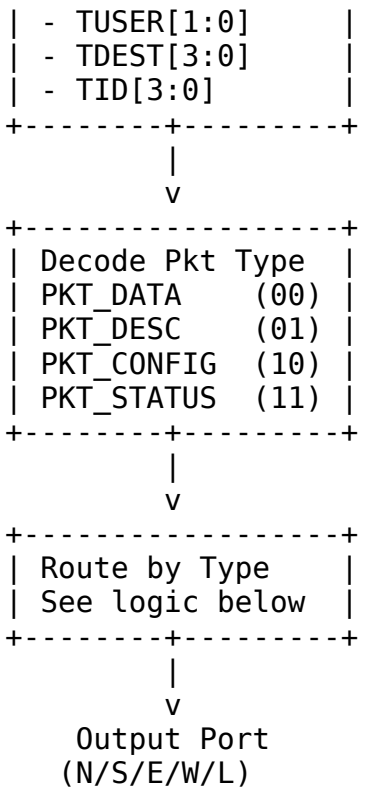
3. Packet Classifier

3.1 Purpose

The Packet Classifier examines packet headers (TUSER, TDEST) and determines the appropriate output port based on the configured routing algorithm.

3.2 Classification Flow

```
Packet arrives at input buffer
         |
         v
+------------------+
| Extract Header   |
```



3.3 Routing Logic by Packet Type

PKT_DATA (0b00)

Route to compute tile or RAPIDS based on TDEST:

```
if (pkt_tdest == 4'd16) begin
    // Destination: RAPIDS DMA (virtual tile 16)
    route_to_virtual_tile(16);  // Via tile 12 south port
end else if (pkt_tdest < 4'd16) begin
    // Destination: Physical tile 0-15
    route_via_xy_algorithm(current_tile, pkt_tdest);
end else begin
    // Invalid destination
    drop_packet();
    set_error_flag(ERR_INVALID_DEST);
end
```

PKT_DESC (0b01)

Always route to RAPIDS (only valid destination):

```
// PKT_DESC always goes to RAPIDS (virtual tile 16)
route_to_virtual_tile(16);
```

PKT_CONFIG (0b10)

Route to tile(s) based on tile mask in payload:

```
// Extract tile mask from payload
logic [31:0] tile_mask = pkt_data[127:96];

// Check if this tile is targeted
if (tile_mask[MY_TILE_ID] == 1'b1) begin
    // Accept locally
    output_port = PORT_LOCAL;

    // Also continue forwarding if other tiles targeted
    if (tile_mask & ~(1 << MY_TILE_ID)) begin
        // Determine next hop via XY routing
        // to reach remaining tiles in mask
        continue_forwarding();
    end
end else begin
    // Not for this tile, forward to next hop
    route_via_xy_algorithm(current_tile, next_tile_in_mask);
end
```

PKT_STATUS (0b11)

Always route to HIVE-C monitoring aggregator:

```
// PKT_STATUS always goes to HIVE-C (virtual tile 17)
route_to_virtual_tile(17);
```

3.4 XY Routing Implementation

```
function [2:0] xy_route(
    input [3:0] current,
    input [3:0] destination
);
    logic [1:0] curr_x, curr_y;
    logic [1:0] dest_x, dest_y;

    // Extract coordinates
    curr_x = current[1:0];     // Column
    curr_y = current[3:2];     // Row

    dest_x = destination[1:0]; // Dest column
    dest_y = destination[3:2]; // Dest row

    // X-dimension first (deadlock-free)
    if (dest_x < curr_x) begin
        return PORT_WEST;
    end else if (dest_x > curr_x) begin
```

```
        return PORT_EAST;
    end else if (dest_y < curr_y) begin
        return PORT_NORTH;
    end else if (dest_y > curr_y) begin
        return PORT_SOUTH;
    end else begin
        return PORT_LOCAL;  // Arrived
    end
endfunction
```

3.5 Virtual Tile Handling

Special logic for virtual tiles (RAPIDS=16, HIVE-C=17):

```
function [2:0] route_to_virtual_tile(
    input [3:0] virtual_id
);
    if (virtual_id == 16) begin
        // RAPIDS: Route to tile 12, then south
        if (MY_TILE_ID == 12) begin
            return PORT_SOUTH;  // Exit to RAPIDS
        end else begin
            return xy_route(MY_TILE_ID, 12);  // Route to tile 12
        end
    end else if (virtual_id == 17) begin
        // HIVE-C: Route to tile 3, then north
        if (MY_TILE_ID == 3) begin
            return PORT_NORTH;  // Exit to HIVE-C
        end else begin
            return xy_route(MY_TILE_ID, 3);  // Route to tile 3
        end
    end
endfunction
```

3.6 Performance

- **Classification latency:** 1 cycle (combinational + 1 FF stage)
- **Complexity:** O(1) - constant time for all packet types
- **Resources:** ~200 LUTs, ~150 FFs per router

Next: Virtual Channel Allocator

4. Virtual Channel Allocator

4.1 Virtual Channel Concept

Two virtual channels (VCs) per physical link prevent protocol-level deadlock and provide QoS:

VC0 - High Priority: - PKT_DESC, PKT_CONFIG, PKT_STATUS - Control/monitoring traffic has priority - Lower buffer depth (4 flits) - less latency critical

VC1 - Standard Priority: - PKT_DATA - Bulk data transfers - Larger buffer depth (8 flits) - more throughput critical

4.2 Credit-Based Flow Control

Each router uses credit-based flow control to prevent buffer overflow:

Initialization

- Downstream router has 8-flit input buffer per VC
- Upstream router initialized with 8 credits per VC
- Credits represent available buffer slots

Operation

```
1. Upstream wants to send flit
   - Check if credits[vc] > 0
   - If yes: send flit, decrement credits[vc]
   - If no: stall (wait for credit return)

2. Downstream receives flit
   - Store in input buffer
   - Process/forward flit
   - Send credit back upstream (increment credits[vc])

3. Steady state:
   - Credits flow opposite to flits
   - No overflow possible (guaranteed by credits)
   - Backpressure propagates naturally
```

4.3 Arbitration Logic

VC0 has strict priority over VC1:

```
// VC0 has strict priority over VC1
if (vc0_has_packet && vc0_credits > 0) begin
    grant_vc0();
end else if (vc1_has_packet && vc1_credits > 0) begin
```

```
    grant_vc1();
end else begin
    idle();
end
```

4.4 Credit Management

```
// Per output port, per VC
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        vc0_credits <= VC0_BUFFER_DEPTH;  // Initialize to 4
        vc1_credits <= VC1_BUFFER_DEPTH;  // Initialize to 8
    end else begin
        // Flit sent: decrement credits
        if (send_flit_vc0) begin
            vc0_credits <= vc0_credits - 1;
        end
        if (send_flit_vc1) begin
            vc1_credits <= vc1_credits - 1;
        end

        // Credit return: increment credits
        if (credit_return_vc0) begin
            vc0_credits <= vc0_credits + 1;
        end
        if (credit_return_vc1) begin
            vc1_credits <= vc1_credits + 1;
        end
    end
end
```

4.5 VC Assignment by Packet Type

```
always_comb begin
    case (pkt_tuser)
        PKT_DATA:   vc_select = VC1;  // Standard priority
        PKT_DESC:   vc_select = VC0;  // High priority
        PKT_CONFIG: vc_select = VC0;  // High priority
        PKT_STATUS: vc_select = VC0;  // High priority
    endcase
end
```

4.6 Performance Characteristics

- **Arbitration latency:** 1 cycle
- **Credit loop latency:** 2-3 cycles (depends on wire delay)
- **Maximum throughput:** 100% (non-blocking when credits available)
- **Resource cost:** ~150 LUTs, ~100 FFs per router

Next: Crossbar Switch

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5. Crossbar Switch

5.1 Non-Blocking 5×5 Design

The crossbar provides conflict-free connections between any input-output port pair:

```
        N    S    E    W    L  (inputs)
      +---+---+---+---+---+
   N  | X | X | X | X | X |
      +---+---+---+---+---+
   S  | X | X | X | X | X |
      +---+---+---+---+---+
   E  | X | X | X | X | X |
      +---+---+---+---+---+
   W  | X | X | X | X | X |
      +---+---+---+---+---+
   L  | X | X | X | X | X |
      +---+---+---+---+---+
     (outputs)

Each X is a 128-bit mux + control
```

5.2 Crossbar Architecture

```
Input Port 0 (North)  ----+
                          |
Input Port 1 (South)  ----+
                          |
Input Port 2 (East)   ----+---- 5:1 Mux ----> Output Port 0
                          |
Input Port 3 (West)   ----+
                          |
Input Port 4 (Local)  ----+

(Repeat for each output port)
```

5.3 Control Logic

```
// Crossbar control signals
logic [4:0][2:0] xbar_sel;  // 5 outputs × 3-bit select (0-4)

// Example: Route North input to East output
xbar_sel[PORT_EAST] = 3'd0;  // Select input 0 (North)

// Multiplexer implementation
always_comb begin
    case (xbar_sel[output_port])
        3'd0: output_data[output_port] = input_data[PORT_NORTH];
        3'd1: output_data[output_port] = input_data[PORT_SOUTH];
        3'd2: output_data[output_port] = input_data[PORT_EAST];
        3'd3: output_data[output_port] = input_data[PORT_WEST];
        3'd4: output_data[output_port] = input_data[PORT_LOCAL];
    endcase
end
```

5.4 Conflict Resolution

No conflicts possible: Each output port has independent control, and the VC allocator ensures only one input requests a specific output per cycle.

Simultaneous transfers:

```
Example: 4 packets routed simultaneously

Input North -> Output East   (Packet A)
Input South -> Output West   (Packet B)
Input East  -> Output Local  (Packet C)
Input West  -> Output North  (Packet D)
Input Local -> Output South  (Packet E)

All 5 transfers happen in parallel (non-blocking)
```

5.5 Performance

- **Latency:** 1 cycle (combinational mux + output register)
- **Throughput:** 5 × 128 bits @ 100 MHz = 8 GB/s aggregate per router
- **Resource cost:** ~400 LUTs (5 × 5:1 muxes for 128 bits), ~200 FFs

5.6 Implementation Trade-offs

Approach	LUTs	Latency	Notes
Full 5×5 muxes	~400	1 cycle	Used in DELTA
Partial crossbar	~250	1 cycle	Restricts some paths

Approach	LUTs	Latency	Notes
Staged crossbar	~300	2 cycles	Lower LUT cost, higher latency

DELTA uses full 5×5: Maximum flexibility, single-cycle latency critical for low-latency routing.

Back to: Block Overview | **Next Chapter:** AXIS Interface Specification

---

1. AXIS Interface Specification

1.1 AXIS Signal Mapping

All network links use AXIS with these signals:

```
// 128-bit data path
logic [127:0] TDATA;   // Payload (flit data)
logic [15:0]  TKEEP;   // Byte enables (usually all 1's)
logic         TLAST;   // End of packet marker
logic [1:0]   TUSER;   // Packet type (PKT_DATA/DESC/CONFIG/STATUS)
logic [3:0]   TID;     // Source tile ID (for PKT_DATA) or
priority/sequence
logic [3:0]   TDEST;   // Destination tile (0-17)
logic         TVALID;  // Data valid
logic         TREADY;  // Receiver ready (backpressure)
```

1.2 TID Field Usage

CRITICAL: The TID field serves different purposes based on packet type:

Packet Type	TID Meaning	Set By	Preserved By Router
PKT_DATA (to RAPIDS)	Source Tile ID (0-15)	Tile NI (enforced)	Yes - never modified

Packet Type	TID Meaning	Set By	Preserved By Router
PKT_DATA (to Tiles)	Priority/ Sequence	RAPIDS MM2S	Yes
PKT_DESC	Priority Level	HIVE-C	Yes
PKT_CONFIG	Broadcast ID	HIVE-C/ SERV	Yes
PKT_STATUS	Reporting Tile ID	SERV (= MY_TILE_ ID)	Yes

Router TID Handling

Routers NEVER modify TID field - it flows end-to-end from source to destination:

```
// Routers NEVER modify TID field
// It flows end-to-end from source to destination
always_ff @(posedge clk) begin
    if (input_valid && output_ready) begin
        output_tid <= input_tid;  // Pass through unchanged
    end
end
```

1.3 Multi-Flit Packets

Single-Flit Packet (<=128 bits)

```
Beat 0: TDATA[127:0] = payload
        TLAST = 1
        TUSER = packet type
        TDEST = destination
```

Multi-Flit Packet (>128 bits)

```
Beat 0: TDATA[127:0] = payload[127:0]
        TLAST = 0

Beat 1: TDATA[127:0] = payload[255:128]
        TLAST = 0

Beat N: TDATA[127:0] = payload[...]
        TLAST = 1  (final beat)

Note: TUSER, TDEST carried only on first beat
      Router caches for subsequent beats
```

1.4 AXIS Handshaking

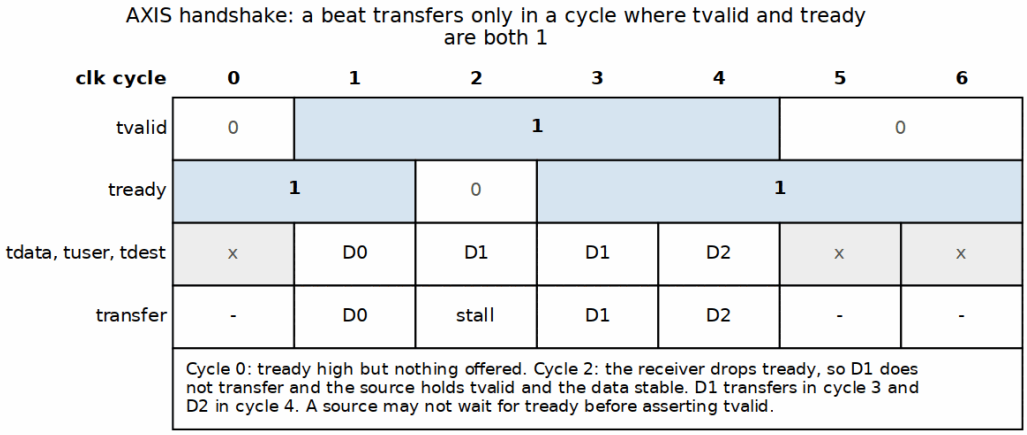
Standard AXIS ready/valid protocol:

```
// Transfer occurs when both valid and ready are high
always_ff @(posedge clk) begin
    if (tvalid && tready) begin
        // Data transfer completes
        // Receiver captures tdata, tuser, tdest, etc.
    end
end

// Backpressure: Receiver asserts tready=0 to stall
// Source must hold tvalid=1 and all data stable until tready=1
```

1.5 Timing Diagram

Figure 3.1: AXIS Handshake With One Stall Cycle



AXIS Handshake With One Stall Cycle

Next: Packet Format and Framing

2. Packet Format and Framing

2.1 Packet Examples

PKT_DATA (Activation Transfer from RAPIDS to Tile)

```
TDATA  = {activation_data}  (128 bits per beat)
TKEEP  = 16'hFFFF           (all bytes valid)
TLAST  = 1                  (single flit, or =1 on final)
TUSER  = 2'b00              (PKT_DATA)
TID    = 4'd2               (priority level - set by RAPIDS MM2S)
TDEST  = 4'd5               (destination: tile 5)
```

PKT_DATA (Results from Tile 7 back to RAPIDS)

```
TDATA  = {result_data}      (128 bits per beat)
TKEEP  = 16'hFFFF
TLAST  = 1
TUSER  = 2'b00              (PKT_DATA)
TID    = 4'd7               (CRITICAL: source tile ID - set by Tile 7
NI)
TDEST  = 4'd16              (destination: RAPIDS virtual tile)

// RAPIDS receives this and uses TID=7 to route to S2MM Channel 7
```

PKT_DESC (DMA Descriptor from HIVE-C)

```
Beat 0:
TDATA  = descriptor[127:0]
TLAST  = 0
TUSER  = 2'b01              (PKT_DESC)
TID    = 4'd0               (high priority)
TDEST  = 4'd16              (destination: RAPIDS)

Beat 1:
TDATA  = descriptor[255:128]
TLAST  = 1
```

PKT_CONFIG (Broadcast to All Tiles)

```
TDATA  = {tile_mask[31:0], command[31:0], payload[63:0]}
TLAST  = 1
TUSER  = 2'b10              (PKT_CONFIG)
TID    = 4'd0               (broadcast identifier)
TDEST  = 4'd15              (doesn't matter for broadcast)
                             (tile_mask determines actual targets)
```

PKT_STATUS (SERV Report from Tile 3 to HIVE-C)

```
TDATA  = {tile_id[7:0], status_type[7:0], timestamp[47:0], data[63:0]}
TLAST  = 1
TUSER  = 2'b11              (PKT_STATUS)
TID    = 4'd3               (reporting tile ID - matches
```

```
TDATA[127:120])
TDEST  = 4'd17                (destination: HIVE-C)
```

2.2 Payload Structure by Packet Type

PKT_DATA Payload

```
TDATA[127:0] = data_payload
    [127:0]  : Actual data (activations, weights, results)
```

PKT_DESC Payload (256 bits = 2 flits)

```
Beat 0 TDATA[127:0]:
    [63:0]   : source_address (DDR address for MM2S)
    [127:64] : dest_address (DDR address for S2MM)

Beat 1 TDATA[127:0]:
    [31:0]   : length (bytes to transfer)
    [35:32]  : dest_tile_id or source_tile_id
    [39:36]  : operation (MM2S=0, S2MM=1)
    [127:40] : reserved/control
```

PKT_CONFIG Payload

```
TDATA[127:0]:
    [31:0]   : tile_mask (32 bits, one per potential tile)
    [63:32]  : command_code
    [127:64] : command_payload
```

PKT_STATUS Payload

```
TDATA[127:0]:
    [7:0]    : tile_id (reporting tile)
    [15:8]   : status_type (congestion, error, periodic)
    [63:16]  : timestamp (48-bit cycle counter)
    [127:64] : status_data (type-specific)
```

Next: External Entity Integration

---

3. External Entity Integration

3.1 RAPIDS and HIVE-C Connections

RAPIDS and HIVE-C are not in the mesh but have dedicated connections:

```
Tile Layout:
      HIVE-C (virtual 17)
         ^
         | (north port)
    +----+----+----+----+
    | T0 | T1 | T2 | T3 |
    +----+----+----+----+
    | T4 | T5 | T6 | T7 |
    +----+----+----+----+
    | T8 | T9 | T10| T11|
    +----+----+----+----+
    | T12| T13| T14| T15|
    +----+----+----+----+
         | (south port)
         v
    RAPIDS (virtual 16)
```

3.2 RAPIDS (Virtual Tile 16)

Connection: - Physically connected to Router 12 (tile 12) south port - All PKT_DESC packets routed to tile 12 first, then south - PKT_DATA from RAPIDS enters at tile 12, routes normally

Interface Signals:

```
// RAPIDS -> DELTA (injection)
logic [127:0] rapids_tx_tdata;
logic [15:0]  rapids_tx_tkeep;
logic         rapids_tx_tlast;
logic [1:0]   rapids_tx_tuser;   // Always PKT_DATA (2'b00)
logic [3:0]   rapids_tx_tid;     // Priority from descriptor
logic [3:0]   rapids_tx_tdest;   // Target tile (0-15)
logic         rapids_tx_tvalid;
logic         rapids_tx_tready;  // From Router 12

// DELTA -> RAPIDS (reception)
logic [127:0] rapids_rx_tdata;
logic [15:0]  rapids_rx_tkeep;
logic         rapids_rx_tlast;
logic [1:0]   rapids_rx_tuser;   // PKT_DATA or PKT_DESC
logic [3:0]   rapids_rx_tid;     // Source tile ID (for PKT_DATA)
logic [3:0]   rapids_rx_tdest;   // Always 16 (RAPIDS)
```

```
logic         rapids_rx_tvalid;
logic         rapids_rx_tready;  // From RAPIDS
```

Routing to RAPIDS:

```
// Any tile sending to TDEST=16 routes to tile 12
// Example: Tile 7 -> Tile 11 -> Tile 15 -> Tile 14 -> Tile 13 -> Tile
12 -> RAPIDS
```

3.3 HIVE-C (Virtual Tile 17)

Connection: - Physically connected to Router 3 (tile 3) north port - All PKT_STATUS packets routed to tile 3 first, then north - PKT_DESC/CONFIG from HIVE-C enters at tile 3, routes normally

Interface Signals:

```
// HIVE-C -> DELTA (injection)
logic [127:0] hive_tx_tdata;
logic [15:0]  hive_tx_tkeep;
logic         hive_tx_tlast;
logic [1:0]   hive_tx_tuser;   // PKT_DESC (2'b01) or PKT_CONFIG
(2'b10)
logic [3:0]   hive_tx_tid;     // Priority or broadcast ID
logic [3:0]   hive_tx_tdest;   // Target tile or 16 (RAPIDS for DESC)
logic         hive_tx_tvalid;
logic         hive_tx_tready;  // From Router 3

// DELTA -> HIVE-C (reception)
logic [127:0] hive_rx_tdata;
logic [15:0]  hive_rx_tkeep;
logic         hive_rx_tlast;
logic [1:0]   hive_rx_tuser;   // Always PKT_STATUS (2'b11)
logic [3:0]   hive_rx_tid;     // Reporting tile ID
logic [3:0]   hive_rx_tdest;   // Always 17 (HIVE-C)
logic         hive_rx_tvalid;
logic         hive_rx_tready;  // From HIVE-C
```

Routing to HIVE-C:

```
// Any tile sending to TDEST=17 routes to tile 3
// Example: Tile 12 -> Tile 8 -> Tile 4 -> Tile 0 -> Tile 1 -> Tile 2
-> Tile 3 -> HIVE-C
```

3.4 Special Routing Cases

Broadcast Configuration

PKT_CONFIG packets can target multiple tiles simultaneously:

```
// Packet payload contains 32-bit tile mask
logic [31:0] tile_mask = pkt_data[127:96];

// Each router checks if it's targeted
if (tile_mask[my_tile_id] == 1'b1) begin
    // Accept packet locally
    forward_to_local_config();

    // Also forward to next hop if other tiles targeted
    if (tile_mask & ~(1 << my_tile_id)) begin
        continue_forwarding();
    end
end
```

Example: Configure all tiles in row 2:

```
tile_mask = 32'h0000_0F00  // Bits 8-11 set (tiles 8-11)
Each router 8-11 accepts locally and forwards
```

Peer-to-Peer Data

Tiles can send PKT_DATA directly to each other:

```
Scenario: Tile 3 sends result to Tile 10

Tile 3 coords: col=3, row=0
Tile 10 coords: col=2, row=2

Step 1: X-dimension (col 3 -> col 2): go WEST
  R3 sends to R2
Step 2: X-dimension complete (col 2 == col 2)
Step 3: Y-dimension (row 0 -> row 2): go SOUTH
  R2 sends to R6 (row 1)
  R6 sends to R10 (row 2)
Step 4: Arrived at R10, send to local port

Total hops: 3 (R3->R2, R2->R6, R6->R10)
Latency: 3 hops × 3 cycles = 9 cycles (mesh mode)
```

Back to: AXIS Interface Specification | **Next Chapter:** Virtual Configuration Contexts

1. Virtual Configuration Contexts

1.1 Supported Topologies

DELTA supports multiple routing modes via virtual contexts:

Context 0 - Mesh (XY Routing)

- Standard 2D mesh with XY dimension-ordered routing
- Default mode, always available
- Deadlock-free by construction

Context 1 - Systolic

- Direct nearest-neighbor communication only
- Router bypass for lowest latency (1-2 cycles/hop)
- North->South and West->East data flow
- Used for matrix multiplication, convolution

Context 2 - Tree Reduction

- Hierarchical aggregation topology
- Bottom row (tiles 12-15) as leaf nodes
- Top row (tiles 0-3) as aggregators
- Optimized for global reduction operations (sum, max)

Context 3 - Custom/Debug

- User-programmable routing tables
- Supports arbitrary topologies
- Enables experimentation with novel routing algorithms

1.2 Context Characteristics

Context	Latency	Throughput	Use Case
Mesh (0)	Medium (3-24 cycles)	High	General-purpose
Systolic (1)	Low (1-2 cycles/hop)	Very High	Matrix ops
Tree (2)	Variable	Medium	Reductions
Custom (3)	User-defined	User-defined	Research

Next: Context Switching

---

2. Context Switching Mechanism

2.1 Switching Sequence

```
1. HIVE-C issues CONFIG_PREPARE command (PKT_CONFIG broadcast)
   - All tiles flush in-flight packets
   - Routers enter quiescent state
   - FIFOs drain (5-10 cycles)

2. HIVE-C broadcasts SET_ROUTING_MODE with context ID
   - Mode register updated: MODE_SELECT <- context_id
   - Routing table pointer switched
   - All routers update simultaneously (1 cycle)

3. HIVE-C issues CONFIG_ACTIVATE command
   - Tiles resume packet injection
   - Network operational in new mode

Total switching time: 10-20 cycles (deterministic)
```

2.2 Example: Switch from Mesh to Systolic

```
// HIVE-C firmware code
void switch_to_systolic_mode(void) {
    // Step 1: Prepare for context switch
    pkt_config_t prep_pkt;
    prep_pkt.tile_mask = 0xFFFF;  // All 16 tiles
    prep_pkt.command = CMD_CONFIG_PREPARE;
    send_config_packet(&prep_pkt);

    // Wait for quiescence
    delay_cycles(10);

    // Step 2: Set new routing mode
    pkt_config_t mode_pkt;
    mode_pkt.tile_mask = 0xFFFF;
    mode_pkt.command = CMD_SET_ROUTING_MODE;
    mode_pkt.payload = CONTEXT_SYSTOLIC;  // Context 1
```

```
    send_config_packet(&mode_pkt);

    // Single-cycle update
    delay_cycles(1);

    // Step 3: Activate network
    pkt_config_t act_pkt;
    act_pkt.tile_mask = 0xFFFF;
    act_pkt.command = CMD_CONFIG_ACTIVATE;
    send_config_packet(&act_pkt);

    // Network now in systolic mode
}
```

2.3 Context Switch Timing

```
Cycle 0:   PREPARE command sent
Cycle 1-5: Packets in flight drain
Cycle 6:   Quiescent state reached
Cycle 7:   SET_ROUTING_MODE command sent
Cycle 8:   All routers update MODE_SELECT
Cycle 9:   ACTIVATE command sent
Cycle 10:  Network operational in new mode

Total: 10 cycles (deterministic)
```

Next: Configuration Commands

---

3. Configuration Commands

3.1 Common PKT_CONFIG Commands

```
Command Code: 0x0001 - SET_ROUTING_MODE
  Payload[1:0] = context_id (0-3)

Command Code: 0x0002 - UPDATE_ROUTE_TABLE
  Payload[7:0]  = table_index
  Payload[39:8] = route_entry

Command Code: 0x0004 - SET_PRIORITY_WEIGHTS
```

```
  Payload[7:0]   = vc0_weight
  Payload[15:8]  = vc1_weight

Command Code: 0x0008 - FLUSH_BUFFERS
  No payload

Command Code: 0x0010 - RESET_STATISTICS
  No payload

Command Code: 0x0020 - SET_CONGESTION_THRESHOLD
  Payload[7:0] = threshold (buffer occupancy)

Command Code: 0x0040 - CONFIG_PREPARE
  No payload (prepares for context switch)

Command Code: 0x0080 - CONFIG_ACTIVATE
  No payload (activates after context switch)
```

3.2 Command Packet Structure

```
TDATA[127:0]:
  [31:0]   : tile_mask (which tiles to configure)
  [63:32]  : command_code (see above)
  [127:64] : command_payload (command-specific)

TLAST = 1 (single flit for most commands)
TUSER = 2'b10 (PKT_CONFIG)
TID = broadcast_id or priority
TDEST = ignored (tile_mask determines targets)
```

3.3 Response Handling

Most configuration commands do not generate responses. Status can be polled via:

```
// Read router status register
uint32_t read_router_status(uint8_t tile_id) {
    // Addressed read via control interface
    // OR wait for periodic PKT_STATUS report
}
```

Next: Complete System Flow

---

4. Complete System Flow

4.1 Use Case: HIVE-C loads activations into Tile 7, Tile 7 computes, returns results

```
1. HIVE-C generates MM2S descriptor (load activations)
   - src_addr = 0x1000_0000 (DDR activation buffer)
   - dst_addr = 0 (unused for MM2S)
   - length = 0x1000 (4 KB)
   - dest_tile_id = 7
   - type = MM2S (Memory->Network)

2. HIVE-C injects descriptor:
   TDATA = descriptor (256 bits, 2 beats)
   TUSER = PKT_DESC (2'b01)
   TID = 0 (high priority)
   TDEST = 16 (RAPIDS)

3. DELTA routes PKT_DESC to RAPIDS:
   R3 (HIVE-C entry) -> R2 -> R1 -> R0 -> R4 -> R8 -> R12 -> RAPIDS

4. RAPIDS MM2S engine executes:
   - Reads 0x1000 bytes from DDR starting at 0x1000_0000
   - Formats AXIS packets:
     TDATA = activation_data
     TUSER = PKT_DATA (2'b00)
     TID = 0 (priority from descriptor)
     TDEST = 7 (destination tile)
   - Sends to DELTA

5. DELTA routes PKT_DATA to Tile 7:
   R12 -> R13 -> R14 -> R15 -> R11 -> R7

6. Tile 7 receives activations, performs computation

7. HIVE-C pre-programs S2MM descriptor for results:
   - src_addr = 0 (unused for S2MM)
   - dst_addr = 0x2000_0000 (DDR result buffer for tile 7)
   - length = 0x800 (2 KB results)
   - source_tile_id = 7 (CRITICAL - expect TID=7)
   - type = S2MM (Network->Memory)

8. HIVE-C injects S2MM descriptor:
   TDATA = descriptor
   TUSER = PKT_DESC
```

```
   TDEST = 16 (RAPIDS)

9. RAPIDS receives S2MM descriptor:
   - Reads source_tile_id = 7
   - Queues descriptor on S2MM Channel 7
   - Channel 7 now waiting for data with TID=7

10. Tile 7 completes computation, sends results:
    - Compute element sends to NI
    - NI enforces: TID <- 7 (MY_TILE_ID)
    - Packet created:
      TDATA = result_data
      TUSER = PKT_DATA (2'b00)
      TID = 7 (ENFORCED by NI - source tile ID)
      TDEST = 16 (RAPIDS)

11. DELTA routes PKT_DATA from Tile 7 to RAPIDS:
    R7 -> R11 -> R15 -> R14 -> R13 -> R12 -> RAPIDS
    (TID=7 preserved throughout)

12. RAPIDS S2MM receives packet:
    - Examines TID = 7
    - Routes to S2MM Channel 7
    - Channel 7 has queued descriptor ready
    - Writes 2 KB to DDR at 0x2000_0000
    - Generates completion interrupt

13. HIVE-C receives interrupt, proceeds to next operation
```

4.2 Key Points

- TID=7 set by Tile 7 NI (hardware enforced)
- TID preserved through entire DELTA routing path
- RAPIDS uses TID=7 to select S2MM Channel 7
- Descriptor on Channel 7 specifies DDR write address
- Complete isolation: each tile's results go to separate channel/memory region

4.3 Packet Type Routing Summary

Packet Type	Generated By	Consumed By	Path
PKT_DATA	RAPIDS, Tiles	RAPIDS, Tiles	DELTA mesh (XY routing)

Packet Type	Generated By	Consumed By	Path
PKT_DESC	HIVE-C only	RAPIDS only	DELTA -> R12 -> RAPIDS
PKT_CONFIG	HIVE-C, SERV	Tile routers	DELTA mesh (broadcast or unicast)
PKT_STATUS	SERV only	HIVE-C only	DELTA -> R3 -> HIVE-C

Back to: Configuration Commands | **Next Chapter:** Configuration Register Bank

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

1. Configuration Register Bank

1.1 Register Map

Each router/tile has configuration registers:

```
Base + 0x00: MODE_SELECT       (RW) - Active context [1:0]
Base + 0x04: ROUTE_TABLE_BASE  (RO) - Current routing table address
Base + 0x08: CUSTOM_ROUTE_0    (RW) - User routing entry 0
Base + 0x0C: CUSTOM_ROUTE_1    (RW) - User routing entry 1
     ...
Base + 0x1C: CUSTOM_ROUTE_7    (RW) - User routing entry 7
Base + 0x20: FILTER_ENABLE     (RW) - Enable packet type filtering
Base + 0x24: ALLOWED_TYPES     (RW) - Permitted TUSER values (bitmask)
Base + 0x28: RATE_LIMIT        (RW) - Max injection rate (pkts/1000
cyc)
Base + 0x2C: STATUS            (RO) - Router state, buffer occupancy
Base + 0x30: ERROR_FLAGS       (RW1C) - Protocol violations, overflows
Base + 0x34: PKT_COUNT_TX      (RO) - Packets transmitted (this tile)
Base + 0x38: PKT_COUNT_RX      (RO) - Packets received (this tile)
```

```
Base + 0x3C: VC0_CREDITS       (RO) - Available VC0 credits
Base + 0x40: VC1_CREDITS       (RO) - Available VC1 credits
```

1.2 MODE_SELECT Register (Offset 0x00)

```
Bits [1:0]: context_id
  00 = Mesh (XY routing)
  01 = Systolic (nearest-neighbor)
  10 = Tree (reduction topology)
  11 = Custom (user-defined)

Bits [31:2]: Reserved (write 0)
```

Write: Changes active routing context (takes effect immediately) **Read:** Returns current context ID

1.3 CUSTOM_ROUTE_x Registers (Offsets 0x08-0x1C)

```
Custom routing table entries (8 total):

Bits [2:0]:   output_port (0=N, 1=S, 2=E, 3=W, 4=L)
Bits [6:3]:   dest_tile (0-15)
Bits [8:7]:   packet_type (00=DATA, 01=DESC, 10=CONFIG, 11=STATUS)
Bits [31:9]:  Reserved
```

Use Case: Context 3 (Custom) uses these entries for user-defined routing

1.4 FILTER_ENABLE Register (Offset 0x20)

```
Bit [0]: enable_filter
  0 = Accept all packet types
  1 = Filter based on ALLOWED_TYPES register

Bits [31:1]: Reserved
```

1.5 ALLOWED_TYPES Register (Offset 0x24)

```
Bit [0]: allow_PKT_DATA
Bit [1]: allow_PKT_DESC
Bit [2]: allow_PKT_CONFIG
Bit [3]: allow_PKT_STATUS
Bits [31:4]: Reserved

Default: 0xF (allow all)
```

Note: FILTER_ENABLE must be 1 for this to take effect

Next: Router Status Registers

---

2. Router Status Registers

2.1 STATUS Register (Offset 0x2C)

```
Bits [2:0]:   router_state
  000 = IDLE (no packets)
  001 = ROUTING (active packet routing)
  010 = CONGESTED (buffers >75% full)
  011 = QUIESCENT (context switch prep)
  100 = ERROR (protocol violation detected)

Bits [7:3]:   north_buf_occupancy [0-8] (input buffer)
Bits [12:8]:  south_buf_occupancy [0-8]
Bits [17:13]: east_buf_occupancy [0-8]
Bits [22:18]: west_buf_occupancy [0-8]
Bits [27:23]: local_buf_occupancy [0-8]

Bits [31:28]: Reserved
```

Read-Only: Reflects current router state and buffer occupancy

2.2 ERROR_FLAGS Register (Offset 0x30)

```
Bit [0]: protocol_violation (invalid TUSER/TDEST combination)
Bit [1]: buffer_overflow (attempted write to full buffer)
Bit [2]: unauthorized_type (tile sent disallowed packet type)
Bit [3]: routing_error (destination unreachable)
Bit [4]: credit_underflow (sent flit without credits)
Bit [5]: deadlock_detected (circular dependency in VC allocation)
Bits [31:6]: Reserved

Write-1-to-Clear (W1C): Write 1 to clear error flag
```

Example:

```
// Check for errors
uint32_t errors = read_reg(ERROR_FLAGS);
if (errors & (1 << 0)) {
    // Handle protocol violation
    log_error("Protocol violation on router");
}
```

```
// Clear errors
write_reg(ERROR_FLAGS, errors);  // W1C
```

2.3 Credit Status Registers

VC0_CREDITS Register (Offset 0x3C)

```
Bits [3:0]:   north_vc0_credits [0-4]
Bits [7:4]:   south_vc0_credits [0-4]
Bits [11:8]:  east_vc0_credits [0-4]
Bits [15:12]: west_vc0_credits [0-4]
Bits [19:16]: local_vc0_credits [0-4]
Bits [31:20]: Reserved
```

VC1_CREDITS Register (Offset 0x40)

```
Bits [3:0]:   north_vc1_credits [0-8]
Bits [7:4]:   south_vc1_credits [0-8]
Bits [11:8]:  east_vc1_credits [0-8]
Bits [15:12]: west_vc1_credits [0-8]
Bits [19:16]: local_vc1_credits [0-8]
Bits [31:20]: Reserved
```

Use Case: Monitoring flow control state, detecting congestion

---

Next: Performance Counters

---

3. Performance Counters

3.1 Packet Count Registers

PKT_COUNT_TX Register (Offset 0x34)

```
Bits [31:0]: packet_count_tx (transmitted packets from this tile)

Counter increments on each TLAST sent from local port
Wraps at 2^32 - 1
```

PKT_COUNT_RX Register (Offset 0x38)

```
Bits [31:0]: packet_count_rx (received packets to this tile)
```

```
Counter increments on each TLAST received at local port
Wraps at 2^32 - 1
```

3.2 Performance Monitoring Example

```
// Monitor tile 7 packet rate
void monitor_tile_7(void) {
    uint32_t tx_start = read_reg_tile(7, PKT_COUNT_TX);
    uint32_t rx_start = read_reg_tile(7, PKT_COUNT_RX);

    // Wait 1 second (100M cycles @ 100 MHz)
    delay_cycles(100000000);

    uint32_t tx_end = read_reg_tile(7, PKT_COUNT_TX);
    uint32_t rx_end = read_reg_tile(7, PKT_COUNT_RX);

    uint32_t tx_rate = tx_end - tx_start;  // Packets/second
    uint32_t rx_rate = rx_end - rx_start;  // Packets/second

    printf("Tile 7: TX=%u pkt/s, RX=%u pkt/s\n", tx_rate, rx_rate);
}
```

3.3 Congestion Detection via Status Polling

```
// Check for congestion across all tiles
void detect_congestion(void) {
    for (uint8_t tile = 0; tile < 16; tile++) {
        uint32_t status = read_reg_tile(tile, STATUS);
        uint8_t router_state = status & 0x7;

        if (router_state == 0b010) {  // CONGESTED
            // Extract buffer occupancies
            uint8_t north_occ = (status >> 3) & 0x1F;
            uint8_t south_occ = (status >> 8) & 0x1F;
            // ... other directions ...

            printf("Tile %u congested: N=%u S=%u\n", tile, north_occ,
south_occ);

            // Take action: reduce injection rate, reroute traffic,
etc.
        }
    }
}
```

3.4 SERV Monitor Integration

SERV monitors can read these registers and generate PKT_STATUS reports:

```
// SERV firmware on each tile
void serv_monitor_loop(void) {
    while (1) {
```

```
        // Read local router status
        uint32_t status = read_local_reg(STATUS);
        uint8_t router_state = status & 0x7;

        if (router_state == 0b010) {  // Congested
            // Build status packet
            pkt_status_t pkt;
            pkt.tile_id = MY_TILE_ID;
            pkt.type = STATUS_CONGESTION;
            pkt.timestamp = get_cycle_count();
            pkt.data = status;  // Full status register

            // Inject via AXIS (TUSER=PKT_STATUS, TDEST=17 for HIVE-C)
            inject_status_packet(&pkt);
        }

        // Periodic monitoring
        delay(MONITOR_INTERVAL);
    }
}
```

Back to: Configuration Register Bank | **Next:** Appendix A - Tile Coordinates

---

Appendix A: Tile Coordinate Mapping

A.1 Tile ID to (Column, Row) Mapping

Tile ID to (Column, Row) mapping:

ID	Col	Row	ID	Col	Row
0	0	0	8	0	2
1	1	0	9	1	2
2	2	0	10	2	2
3	3	0	11	3	2
4	0	1	12	0	3
5	1	1	13	1	3
6	2	1	14	2	3
7	3	1	15	3	3

```
Virtual tiles:
16   0    4  (RAPIDS - south of tile 12)
17   3   -1  (HIVE-C - north of tile 3)
```

A.2 Coordinate Extraction

```
// Extract coordinates from tile ID
column = tile_id[1:0];  // Bits [1:0]
row    = tile_id[3:2];  // Bits [3:2]

// Example: Tile 10
// ID = 10 = 4'b1010
// column = 2'b10 = 2
// row    = 2'b10 = 2
// Coordinates: (2, 2)
```

A.3 Reverse Mapping (Coordinates to ID)

```
tile_id = {row[1:0], column[1:0]};

// Example: (column=3, row=1)
// tile_id = {2'b01, 2'b11} = 4'b0111 = 7
```

A.4 Physical Layout

```
         Column
       0   1   2   3
     +---+---+---+---+
  0  | 0 | 1 | 2 | 3 |  Row 0
     +---+---+---+---+
  1  | 4 | 5 | 6 | 7 |  Row 1
     +---+---+---+---+
  2  | 8 | 9 |10 |11 |  Row 2
     +---+---+---+---+
  3  |12 |13 |14 |15 |  Row 3
     +---+---+---+---+
```

A.5 Manhattan Distance Calculation

```
function [3:0] manhattan_distance(
    input [3:0] src_tile,
    input [3:0] dst_tile
);
    logic [1:0] src_x, src_y;
    logic [1:0] dst_x, dst_y;
    logic [1:0] delta_x, delta_y;

    src_x = src_tile[1:0];
    src_y = src_tile[3:2];
    dst_x = dst_tile[1:0];
    dst_y = dst_tile[3:2];
```

```
    delta_x = (dst_x > src_x) ? (dst_x - src_x) : (src_x - dst_x);
    delta_y = (dst_y > src_y) ? (dst_y - src_y) : (src_y - dst_y);

    return delta_x + delta_y;  // Hops required
endfunction

// Example: Tile 0 -> Tile 15
// (0,0) -> (3,3)
// delta_x = 3, delta_y = 3
// distance = 6 hops
```

Back to: Index

Appendix B: Configuration Commands

B.1 Command Code Reference

Code	Command	Payload	Description
0x0001	SET_ROUTING_MODE	context_id[1:0]	Change routing context (mesh/systolic/tree/custom)
0x0002	UPDATE_ROUTE_TABLE	table_idx[7:0], entry[31:0]	Update custom routing table entry
0x0004	SET_PRIORITY_WEIGHTS	vc0_wt[7:0], vc1_wt[7:0]	Adjust VC arbitration weights
0x0008	FLUSH_BUFFERS	none	Drain all router buffers
0x0010	RESET_STATISTICS	none	Clear packet counters
0x0020	SET_CONGESTION_THRESHOLD	threshold[7:0]	Set buffer occupancy threshold for congestion flag
0x0040	CONFIG_PREPAR	none	Prepare for context switch

Code	Command	Payload	Description
	E		(quiesce network)
0x0080	CONFIG_ACTIVATE	none	Resume operation after context switch
0x0100	ENABLE_FILTER	types[3:0]	Enable packet type filtering
0x0200	DISABLE_FILTER	none	Disable packet type filtering

B.2 Broadcast vs Unicast

Broadcast (tile_mask = 0xFFFF): - Reaches all 16 tiles - Used for global configuration (context switch, reset stats)

Multicast (tile_mask = subset): - Reaches specific tiles - Example: Configure row 2 only (mask = 0x0F00)

Unicast (tile_mask = single bit): - Reaches one tile - Example: Update custom route on tile 7 (mask = 0x0080)

B.3 Command Examples

Example 1: Global Context Switch

```
// Switch all tiles to systolic mode
pkt_config_t cmd;
cmd.tile_mask = 0xFFFF;              // All tiles
cmd.command = CMD_SET_ROUTING_MODE;
cmd.payload = CONTEXT_SYSTOLIC;      // Mode 1
send_config_packet(&cmd);
```

Example 2: Configure Row 2 for Higher Priority

```
// Set VC0 weight higher for tiles 8-11
pkt_config_t cmd;
cmd.tile_mask = 0x0F00;              // Tiles 8-11 (row 2)
cmd.command = CMD_SET_PRIORITY_WEIGHTS;
cmd.payload = (8 << 8) | 2;          // VC0=8, VC1=2
send_config_packet(&cmd);
```

Example 3: Update Custom Route on Tile 7

```
// Custom route: Tile 7 sends PKT_DATA to tile 10 via EAST port
pkt_config_t cmd;
cmd.tile_mask = 0x0080;              // Tile 7 only
cmd.command = CMD_UPDATE_ROUTE_TABLE;
cmd.payload = (0 << 8) | (2 << 0) | (10 << 3) | (PKT_DATA << 7);
```

```
//              ^table  ^port E   ^dest 10   ^pkt type
send_config_packet(&cmd);
```

Back to: Index

Appendix C: Performance Characteristics

C.1 Latency Model

Single-Hop Latency

- Input buffering: 1 cycle
- Routing decision: 1 cycle
- Crossbar traversal: 1 cycle
- Output buffering: 1 cycle (optional)
- **Total: 3-4 cycles per hop**

Multi-Hop Examples (4×4 Mesh)

Source -> Dest	Hops	Latency (cycles)	Notes
T0 -> T1	1	3-4	Adjacent (horizontal)
T0 -> T4	1	3-4	Adjacent (vertical)
T0 -> T5	2	6-8	XY: East, then South
T0 -> T15	6	18-24	Worst case (diagonal)
T7 -> T12	5	15-20	Example peer-to-peer
Any -> RAPIDS (avg)	3.5	10-14	Average to virtual tile 16

Source -> Dest	Hops	Latency (cycles)	Notes
Any -> HIVE-C (avg)	3.5	10-14	Average to virtual tile 17

Context-Dependent Latency

Context	Per-Hop Latency	Notes
Mesh (XY)	3-4 cycles	Standard routing
Systolic	1-2 cycles	Router bypass, nearest-neighbor only
Tree	Variable	Depends on position in tree
Custom	User-defined	Configurable via routing tables

C.2 Throughput Analysis

Per-Link Bandwidth

- 128 bits @ 100 MHz = 1.6 GB/s per direction
- Full-duplex: 3.2 GB/s aggregate per link

Bisection Bandwidth (4×4 Mesh)

- Horizontal cuts: 4 links × 1.6 GB/s = 6.4 GB/s
- Vertical cuts: 4 links × 1.6 GB/s = 6.4 GB/s
- **Total: 6.4 GB/s bisection**

Network Aggregate

- 40 total links (24 H + 16 V) in 4×4 mesh
- 40 × 1.6 GB/s = 64 GB/s aggregate
- Actual sustained: 60-70% (38-45 GB/s) due to contention

Per-Tile Injection/Reception

- Maximum: 1.6 GB/s (single local port)
- Practical: 1.2-1.4 GB/s (accounting for flow control)

C.3 Resource Utilization

Per Router (LUTs/FFs/BRAM)

Component	LUTs	FFs	BRAM (36Kb)
Input buffers (5×8 flits)	2,000	1,600	64 bits
Routing logic	200	150	0
VC allocator	150	100	0
Crossbar (5×5)	400	200	0
Output buffers (5×4 flits)	800	600	32 bits
Total per router	**~3,550**	**~2,650**	**~2**

Full 4×4 Network

- 16 routers × 3,550 LUTs = 56,800 LUTs
- 16 routers × 2,650 FFs = 42,400 FFs
- 16 routers × 2 BRAM = 32 BRAM blocks
- **Plus NI overhead: +9,600 LUTs, +6,400 FFs, +4 BRAM**
- **Total DELTA: ~66,400 LUTs, ~48,800 FFs, ~36 BRAM**

FPGA Utilization Examples

FPGA Device	LUTs Available	% Used	FFs Available	% Used	BRAM Available	% Used
Artix-7 XC7A100T	63,400	105%	126,800	38%	135	27%
Kintex-7 XC7K325T	203,800	33%	407,600	12%	445	8%
Virtex-7 XC7VX485T	303,600	22%	607,200	8%	1,030	3%

Note: 4×4 DELTA requires mid-range FPGA (Kintex-7 or higher)

C.4 Power Estimation

Dynamic Power @ 100 MHz

- Routers (16×): ~120 mW each = 1,920 mW

- Network Interfaces (16×): ~30 mW each = 480 mW
- Interconnect wiring: ~200 mW
- **Total: ~2,600 mW (2.6 W) @ 100 MHz**

Scaling with Frequency

Frequency	Est. Power	Notes
50 MHz	~1.3 W	Low-power mode
100 MHz	~2.6 W	Typical operation
200 MHz	~5.5 W	High-performance (if timing met)

C.5 Scalability

Larger Mesh Sizes

Mesh Size	Tiles	Routers	Est. LUTs	Est. Power @ 100MHz
2×2	4	4	~16,600	~0.7 W
4×4	16	16	~66,400	~2.6 W
8×8	64	64	~265,600	~10 W
16×16	256	256	~1,062,400	~40 W

Note: 8×8 and larger require high-end FPGAs or ASICs

Back to: Index

Document Version: 0.3 (Early Draft - Proof of Concept) **Last Updated:** 2025-10-19
Status: Preliminary specification, subject to significant change **Maintained By:** DELTA Development Team